Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

RouteLens: Live-Traffic Multiple Routes Visualization and Optimization System

Team Name: *Team #*	*Carpe Diem* *DSCPP-III-2026-T278*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Upreti, Gunjita – 2510382973* *2510382973@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Rawat, Sneha – 2510012141* *2510012141@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rawat, Aayush – 2510370528* *2510370528@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

RouteLens is a web-based route planning system that helps commuters choose between several driving routes by showing them together on one map and comparing them on more than travel time. For a chosen start point and destination, it generates up to three genuinely different routes over the real road network of Dehradun, draws each route in a distinct colour, and attaches an information box with the estimated time of arrival (ETA), distance, number of traffic signals and a Low/Medium/High traffic level. The recommended route is highlighted with a short explanation of why it was preferred.

Following the Phase-I review, the project no longer uses an imaginary map or simulated traffic. The road graph is built from real OpenStreetMap (OSM) data, and live traffic speeds come from the TomTom Traffic Flow API through API keys stored only on the server. When live data is unavailable, the system falls back to free-flow travel times and informs the user.

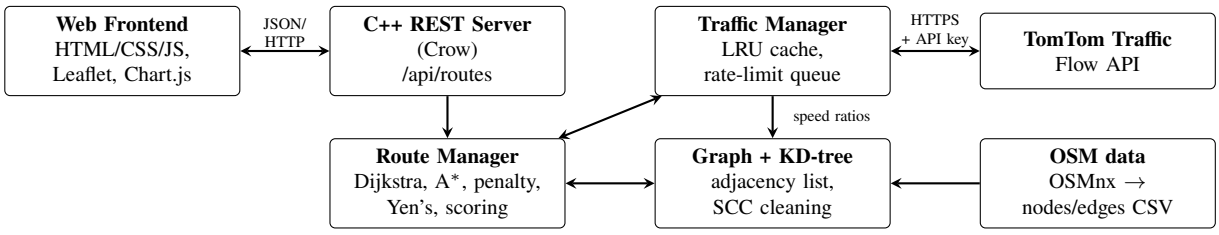
The core is written in C++ using a directed adjacency-list graph, a hand-written binary min-heap for Dijkstra's algorithm and A* search, the penalty method and Yen's algorithm for alternative routes, a KD-tree for nearest-road lookup and an LRU cache for traffic data. Users can adjust their priorities (time, distance, signals, congestion), making the comparison a transparent decision aid rather than a single fixed answer.

Updated Project Approach and Architecture (2 pts)

System design. RouteLens uses a three-tier client–server design. (1) *Offline data layer:* a Python script using OSMnx extracts the drivable OSM network of Dehradun and exports nodes, directed edges (length, road class, free-flow speed), road geometry and traffic-signal tags to CSV files. (2) *C++17 backend:* builds a directed adjacency-list graph (hash map for OSM-ID mapping), keeps the largest strongly connected component (iterative Tarjan's algorithm) and snaps clicked points to the nearest node with a KD-tree. Routes are computed with Dijkstra's algorithm and A* on a self-written binary min-heap; alternatives come from the penalty method and Yen's k-shortest paths, filtered by overlap and stretch, then ranked by a Pareto filter and a user-weighted score. The Traffic Manager samples points along candidate routes, queries the TomTom Traffic Flow Segment Data API and updates the speed ratio of each matched edge, using an LRU cache (hash map + doubly linked list, 5-minute expiry) and a queue-based rate limiter. (3) *Web frontend:* HTML, CSS and JavaScript with Leaflet.js for the map and Chart.js for route comparison.

Communication protocols. JSON over HTTP REST endpoints (`/api/routes`, `/api/health`, `/api/traffic/status`) between browser and server; HTTPS from the server to TomTom. The API key is read from an environment variable and never reaches the browser.

Libraries and tools. Crow (web server), cpp-httplib with OpenSSL (HTTPS client), nlohmann/json, SQLite, GoogleTest, OSMnx, Leaflet.js, Chart.js; CMake, Git/GitHub and GitHub Actions CI.



Tasks Completed (7 pts)

Task Completed	Team Member
OSM data pipeline: OSMnx extraction of the Dehradun road network; export of nodes, directed edges, geometry and signal tags; C++ CSV loader with hash-map ID mapping; largest-SCC cleaning (iterative Tarjan).	Sneha Rawat
Hand-written binary min-heap; Dijkstra's algorithm and A* search with lazy deletion; unit tests against reference shortest-path results.	Gunjita Upreti
Alternative routes: penalty method, Yen's k-shortest paths, overlap and stretch filtering; Pareto filter, weighted scoring and route explanation text.	Gunjita Upreti
KD-tree nearest-node snapping; edge-geometry loading so routes follow real road curves.	Sneha Rawat
TomTom Traffic Flow client with server-side API key; LRU cache with expiry; queue-based rate limiter; segment-to-edge matching; ETA, traffic-level and signal-count computation; free-flow fallback.	Sneha Rawat
Crow REST server (`/api/routes`, `/api/health`, `/api/traffic/status`), JSON input validation and static website hosting.	Aayush Rawat
Website: Leaflet map with draggable start/end markers, colour-coded routes with information boxes, highlighted recommended route, route cards, Chart.js comparison and priority sliders.	Aayush Rawat
GitHub repository, CMake build, GitHub Actions CI, secure API-key handling (`.env` excluded via `.gitignore`); module integration.	Aayush Rawat (repository, CI); Gunjita Upreti (integration)

Challenges/Roadblocks (7 pts)

1. Fitting live traffic onto our own graph. TomTom reports traffic for its own road segments, not for our OSM edges, and querying every edge is impossible within the free limit of 2,500 non-tile requests per day. We sample points every 300–500 m along candidate routes on major roads and match each returned segment to graph edges within about 20 m and in a similar direction, using the KD-tree. An area-wide refresh with HERE's bounding-box flow API is planned to reduce calls further.

2. API quota. An LRU cache with 5-minute expiry, removal of duplicate sample points and a queue-based rate limiter keep usage low; daily calls and cache hit rate are monitored through `/api/traffic/status`.

3. Incomplete real-world data. Live coverage on minor roads is limited and many traffic signals are not tagged in OSM. Live data is applied only where the API's confidence is adequate, with a visible free-flow fallback elsewhere; signals on test routes are audited through a local overrides file.

4. Near-duplicate alternatives. Yen's algorithm tends to return routes differing only by a small detour, so the penalty method is the primary generator, with overlap and stretch filters on all candidates; thresholds will be tuned during evaluation.

5. Engineering issues. Tarjan's algorithm is written iteratively to avoid stack overflow on a large graph; a mutex and per-request weight copies prevent data races in the multithreaded server; the fixed signal delay is applied only in free-flow mode so that waiting time already included in live speeds is not counted twice.

Tasks Pending (7 pts)

Task Pending	**Team Member (to complete the task)**
Scenario (what-if) mode: block or slow selected roads, BFS reachability check, `/api/scenario` endpoint and its user interface.	Gunjita Upreti (backend); Aayush Rawat (interface)
Traffic history logging in SQLite and departure-time comparison of routes.	Sneha Rawat
Area-wide traffic refresh using the HERE Traffic API v7 (bounding-box flow) to reduce per-query API calls.	Sneha Rawat
Algorithm evaluation: correctness on 1,000 random origin–destination pairs against NetworkX; Dijkstra vs. A*; own heap vs. `std::priority_queue`; penalty method vs. Yen's (overlap, stretch).	Gunjita Upreti
ETA accuracy study against TomTom Routing ETAs and real timed trips; failure analysis.	Sneha Rawat
Usability study with classmates; mobile layout and colour-blind accessibility; Docker-based deployment.	Aayush Rawat
Final report, documentation, demo video and viva preparation.	All members

Project Outcome/Deliverables (2 pts)

The final deliverable is a working RouteLens website backed by a C++ routing engine, demonstrated on the real road network of Dehradun with live traffic data.

1. Web application: selection of start and destination by clicking or dragging markers; up to three colour-coded routes on one map, each with an information box (ETA, distance, traffic signals, traffic level); a highlighted recommended route with an explanation; adjustable priorities; and a what-if scenario mode.

2. C++ routing engine: hand-written data structures (adjacency-list graph, binary min-heap, KD-tree, LRU cache, queue, hash map) and algorithms (Dijkstra, A*, penalty method, Yen's algorithm, Tarjan's SCC, BFS), exposed through a REST API.

3. Live-traffic integration: TomTom traffic data with server-side API keys, caching, rate limiting and free-flow fallback.

4. Evaluation report: correctness, performance, ETA-accuracy and usability results with failure analysis.

5. Source code repository: with unit tests, continuous integration, documentation and a demo video.

Progress Overview (2 pts)

How much is done: About **78%** of the project is complete (Phase-II target: 75%).

What works now: A user can click a start and end point on the website and get up to three routes calculated by our own C++ code, using real Dehradun roads and live TomTom traffic. Each route shows its ETA, distance, number of signals and traffic level, and the best route is highlighted.

Ahead of schedule: Priority sliders and route explanations, which were not in our Phase-I plan, are already working.

Behind schedule: Scenario mode (blocking or slowing a road) is not finished, because matching TomTom traffic data to our roads took longer than expected. It is our first task in Phase III.

Still to do (about 22%): scenario mode, traffic history, testing and evaluation, user study, design polish, deployment and final documentation.

Codebase Information (2 pts)

Repository: [https://github.com/gunjita18/RouteLens-DSCPP].

Branches: `main` (stable, protected; merged only through reviewed pull requests), `dev` (integration) and `feature/*` branches. **Structure:** `preprocess/`, `src/{graph, ds, routing, traffic, api}`, `web/`, `tests/`.

Important commits:
– CMake/Crow project skeleton and GitHub Actions CI
– OSMnx export script and C++ graph loader with SCC cleaning
– Binary min-heap, Dijkstra and A* with unit tests
– Penalty method, Yen's algorithm and route filtering
– TomTom traffic client, LRU cache and rate limiter
– Leaflet frontend with multi-route display and comparison panel

Security: API keys are not stored in the repository; only `.env.example` is committed and `.env` is excluded through `.gitignore`.

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit tests (data structures)	Pass	Heap, KD-tree and cache checked on sample inputs.
Route correctness	Pass	Routes checked against a reference tool on sample trips.
Traffic API integration	Pass	Live traffic data received; free-flow fallback works.
Website (manual testing)	Pass	Routes and info boxes display correctly in the browser.
Performance and accuracy evaluation	Pending	Planned for Phase III.

Deliverables Progress (2 pts)

1. **Website** – *In progress.* Main features are working; scenario mode and design polish are pending.
2. **C++ routing engine** – *Mostly completed.* Scenario mode is pending.
3. **Live-traffic integration** – *Completed.*
4. **Traffic history** – *Pending.*
5. **Evaluation report** – *Pending* (Phase III).
6. **Code repository and documentation** – *In progress.*